

数据库应用系统综合设计实验报告

实验名称： 数据查询 .

实验类型： 设计型实验

指导教师： 陈骏

专业班级： _____

姓 名： _____

学 号： _____

联系电话： _____

成绩： _____

1. 实验目的

- 掌握查询语句的一般格式;
- 熟练掌握单表查询、连接查询、集合查询、统计查询和嵌套查询。

2. 实验环境

oracle Database 19c 企业版、(64 位) PLSQL Developer 12、Windows 10

3. 实验内容

1、查询“红楼梦”目前可借的各图书编号，及所属版本信息。

```
SELECT 图书.图书编号,书目.ISBN,书目.书名,书目.作者,书目.出版单位
```

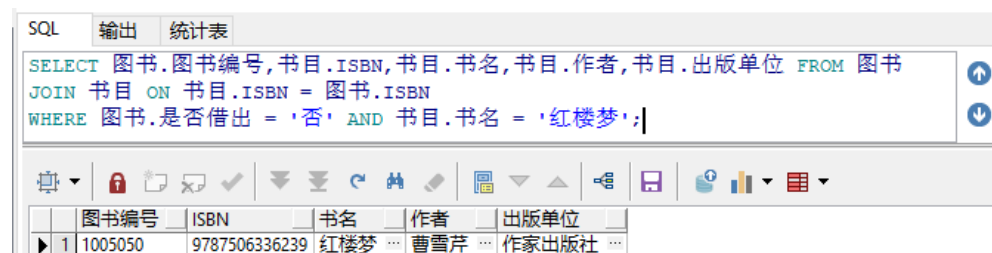
```
FROM 图书
```

```
JOIN 书目 ON 书目.ISBN = 图书.ISBN
```

```
WHERE 图书.是否借出 = '否' AND 书目.书名 = '红楼梦';
```

--使用 JOIN 语句将图书和书目表连接起来，连接条件是两个表中的 ISBN

字段相等。



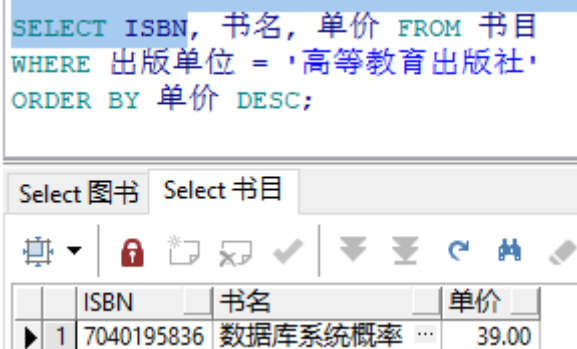
JOIN 语句将两个表连接起来，以便能够同时访问两个表中的数据

2、查找高等教育出版社的所有书目及单价，结果按单价降序排序。

```
SELECT ISBN, 书名, 单价 FROM 书目
```

```
WHERE 出版单位 = '高等教育出版社' --筛选条件
```

```
ORDER BY 单价 DESC;
```



```
SELECT ISBN, 书名, 单价 FROM 书目
WHERE 出版单位 = '高等教育出版社'
ORDER BY 单价 DESC;
```

	ISBN	书名	单价
1	7040195836	数据库系统概率 ...	39.00

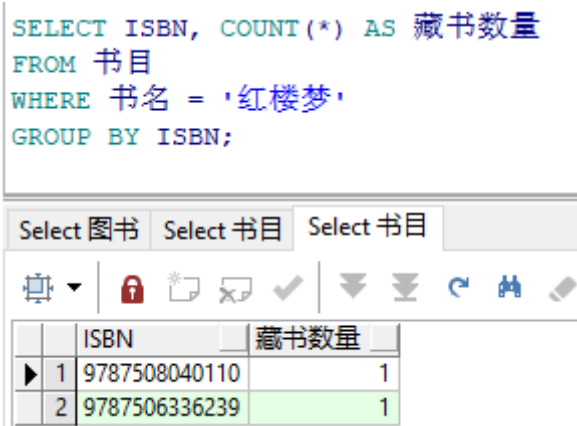
3、统计“红楼梦”各版的藏书数量（ISBN 不同则版本不同）。

```
SELECT ISBN, COUNT(*) AS 藏书数量
```

```
FROM 书目
```

```
WHERE 书名 = '红楼梦'
```

```
GROUP BY ISBN;
```



```
SELECT ISBN, COUNT(*) AS 藏书数量
FROM 书目
WHERE 书名 = '红楼梦'
GROUP BY ISBN;
```

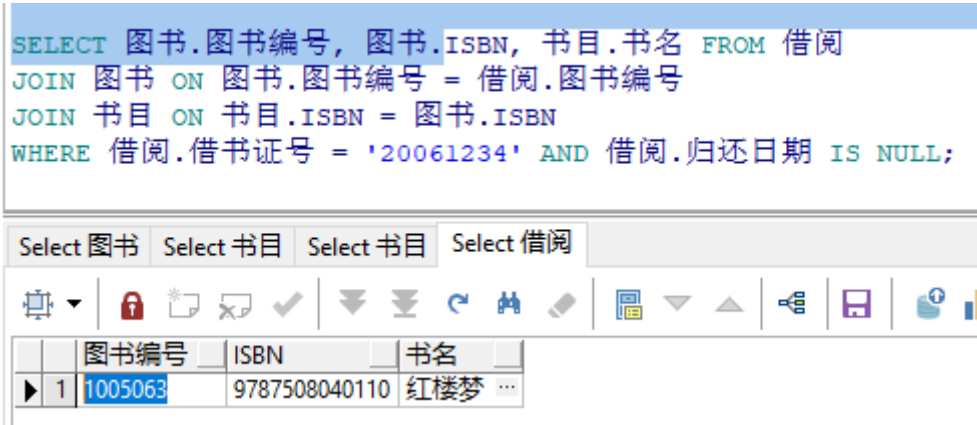
	ISBN	藏书数量
1	9787508040110	1
2	9787506336239	1

- GROUP BY ISBN: QL 引擎会根据 ISBN 字段的值将结果分组，并对每个

组应用 COUNT(*) 函数来计算每组的记录数。

4、查询学号“20061234”号借书证借阅未还的图书的信息。

```
SELECT 图书.图书编号, 图书.ISBN, 书目.书名 FROM 借阅  
  
JOIN 图书 ON 图书.图书编号 = 借阅.图书编号  
  
JOIN 书目 ON 书目.ISBN = 图书.ISBN    --将三个表连接起来  
  
WHERE 借阅.借书证号 = '20061234' AND 借阅.归还日期 IS NULL;
```



5、查询各个出版社的图书最高单价、平均单价。

```
SELECT 出版单位, MAX(单价) AS 最高单价, AVG(单价) AS 平均单价  
  
FROM 书目  
  
GROUP BY 出版单位;
```

<pre>SELECT 出版单位, MAX(单价) AS 最高单价, AVG(单价) AS 平均单价 FROM 书目 GROUP BY 出版单位;</pre>				
Select 图书Select 书目Select 书目Select 借阅Select 书目				
	出版单位	最高单价	平均单价	
1	人民出版社	33.8	26.9	
2	作家出版社	34.3	34.3	
3	高等教育出版社	39	39	

使用 GROUP BY 子句，根据出版社的名称将结果分组，并为每个组分别计算 MAX 和 AVG 聚合函数的结果。

6、要查询借阅了两本和两本以上图书的读者的个人信息。

```
SELECT 读者.借书证号, 读者.姓名, 读者.单位, 读者.性别, 读者.地址, 读者.
联系电话, 读者.身份证编号, COUNT(借阅.借阅流水号) AS 借阅次数
FROM 读者
JOIN 借阅 ON 借阅.借书证号 = 读者.借书证号
GROUP BY 读者.借书证号, 读者.姓名, 读者.单位, 读者.性别, 读者.地址,
读者.联系电话, 读者.身份证编号
HAVING COUNT(借阅.借阅流水号) >= 2;
```

<pre>SELECT 读者.借书证号, 读者.姓名, 读者.单位, 读者.性别, 读者.地址, 读者.联系电话, 读者.身份证编号, COUNT(借阅.借阅流水号) AS 借阅次数 FROM 读者 JOIN 借阅 ON 借阅.借书证号 = 读者.借书证号 GROUP BY 读者.借书证号, 读者.姓名, 读者.单位, 读者.性别, 读者.地址, 读者.联系电话, 读者.身份证编号</pre>							
Select 图书Select 书目Select 书目Select 借阅Select 书目Select 读者							
借书证号	姓名	单位	性别	地址	联系电话	身份证编号	借阅次数
1	20071235	李晓明	四川成都工商银行	男			3

GROUP BY 子句用于将结果集分成多个组，每个组内的行具有相同的列值。

除了聚合函数外，**SELECT 列表中的其他所有列都必须在 GROUP BY 子句中指定。**

7、查询“王菲”的单位、所借图书的书名和借阅日期。

```
SELECT 读者.单位, 书目.书名, 借阅.借书日期 FROM 读者
```

```
JOIN 借阅 ON 借阅.借书证号 = 读者.借书证号
```

```
JOIN 图书 ON 借阅.图书编号 = 图书.图书编号
```

```
JOIN 书目 ON 书目.ISBN = 图书.ISBN
```

```
WHERE 读者.姓名 = '王菲';
```

```
SELECT 读者.单位, 书目.书名, 借阅.借书日期 FROM 读者
JOIN 借阅 ON 借阅.借书证号 = 读者.借书证号
JOIN 图书 ON 借阅.图书编号 = 图书.图书编号
JOIN 书目 ON 书目.ISBN = 图书.ISBN
WHERE 读者.姓名 = '王菲';
```

	单位	书名	借书日期
1	四川绵阳西科大计算机学院 ...	心学之路 ...	2011/09/10

8、查询每类图书的册数和平均单价。

```
SELECT 图书分类号, COUNT(*) AS 册数, AVG(单价) AS 平均单价
```

```
FROM 书目
```

```
GROUP BY 图书分类号;
```

```
SELECT 图书分类号, COUNT(*) AS 册数, AVG(单价) AS 平均单价
FROM 书目
GROUP BY 图书分类号;
```

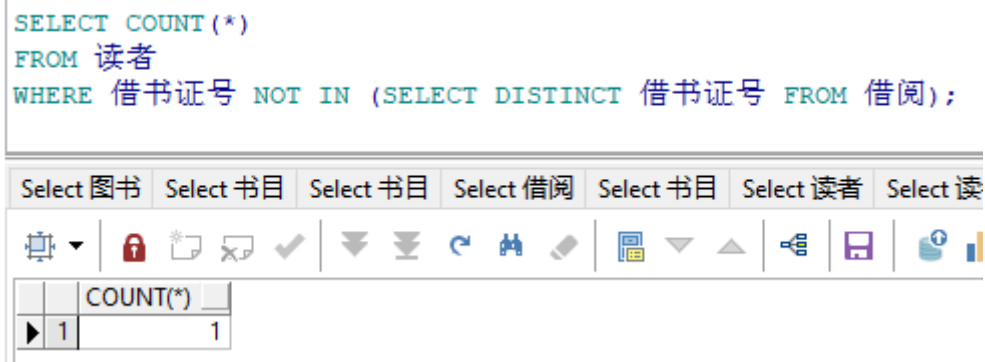
	图书分类号	册数	平均单价
1	300	1	33.8
2	200	1	39
3	100	2	27.15

9、统计从未借书的读者人数。

```
SELECT COUNT(*)
```

```
FROM 读者
```

```
WHERE 借书证号 NOT IN (SELECT DISTINCT 借书证号 FROM 借阅);
```

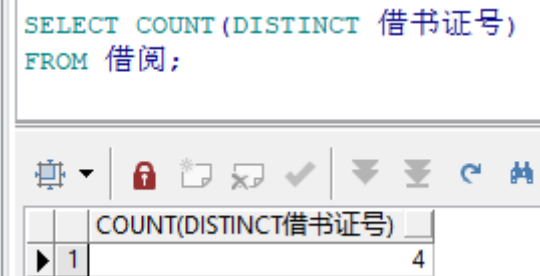


SELECT DISTINCT 借书证号 FROM 借阅从借阅表中选择所有不同的借书证号。DISTINCT 关键字确保每个借书证号只被计算一次，即使同一个读者有多条借阅记录，也只计算一次。

10、统计参与借书的人数。

```
SELECT COUNT(DISTINCT 借书证号)
```

```
FROM 借阅;
```



11、 找出所有借书未还的读者的信息及所借图书编号及名称。

```
SELECT 读者.借书证号, 读者.姓名,读者.单位, 读者.性别, 读者.地址, 读者.
联系电话, 读者.身份证编号, 图书.图书编号, 书目.书名

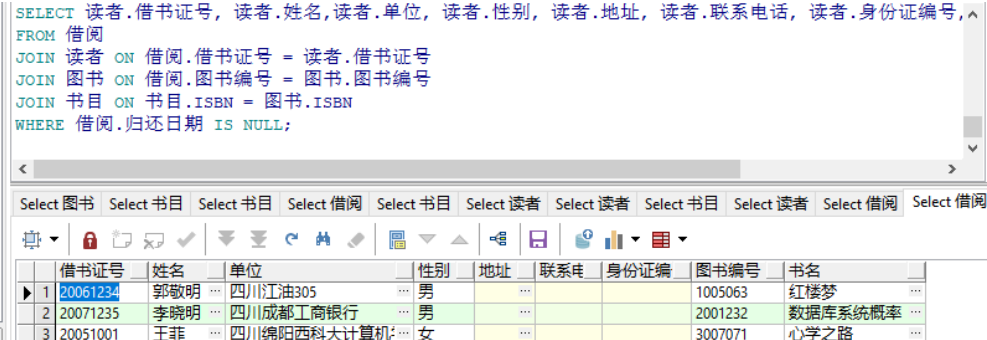
FROM 借阅

JOIN 读者 ON 借阅.借书证号 = 读者.借书证号

JOIN 图书 ON 借阅.图书编号 = 图书.图书编号

JOIN 书目 ON 书目.ISBN = 图书.ISBN

WHERE 借阅.归还日期 IS NULL;
```



The screenshot shows a SQL query window with the following query:

```
SELECT 读者.借书证号, 读者.姓名, 读者.单位, 读者.性别, 读者.地址, 读者.联系电话, 读者.身份证编号, ^
FROM 借阅
JOIN 读者 ON 借阅.借书证号 = 读者.借书证号
JOIN 图书 ON 借阅.图书编号 = 图书.图书编号
JOIN 书目 ON 书目.ISBN = 图书.ISBN
WHERE 借阅.归还日期 IS NULL;
```

The results are displayed in a table with the following columns: 借书证号, 姓名, 单位, 性别, 地址, 联系电话, 身份证编号, 图书编号, 书名.

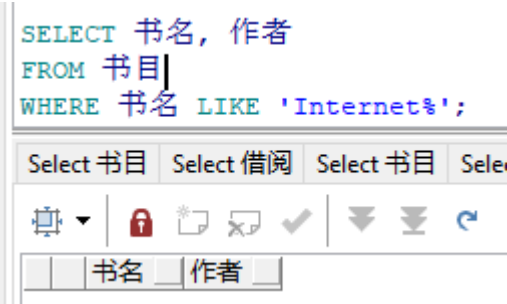
	借书证号	姓名	单位	性别	地址	联系电话	身份证编号	图书编号	书名
1	20061234	郭敬明	四川江油305	男	...	...	...	1005063	红楼梦
2	20071235	李晓明	四川成都工商银行	男	...	...	...	2001232	数据库系统概率
3	20051001	王菲	四川绵阳西科大计算机	女	...	...	...	3007071	心学之路

12、 检索书名是以“Internet”开头的所有图书的书名和作者。

```
SELECT 书名, 作者

FROM 书目

WHERE 书名 LIKE 'Internet%';
```



The screenshot shows a SQL query window with the following query:

```
SELECT 书名, 作者
FROM 书目
WHERE 书名 LIKE 'Internet%';
```

The results are displayed in a table with the following columns: 书名, 作者.

书名	作者
----	----

使用 LIKE 操作符, LIKE 操作符用于在 WHERE 子句中搜索列中的指定模式

的值。通配符 % 来匹配以 "Internet" 开头的书名。% 代表任意数量的字符，这个条件将匹配 "Internet" 后面跟着任意字符串的所有书名。

13、 查询各图书的罚款总数。

```
SELECT 书目.书名,书目.ISBN,COUNT(CASE WHEN 借阅.罚款分类号 IS
NOT NULL THEN 1 END) AS 罚款次数
FROM 借阅
JOIN 图书 ON 借阅.图书编号 = 图书.图书编号
JOIN 书目 ON 书目.ISBN = 图书.ISBN
GROUP BY 书目.ISBN, 书目.书名;
```

	书名	ISBN	罚款次数
1	心学之路	9787010073750	0
2	数据库系统概率	7040195836	0
3	红楼梦	9787508040110	1
4	红楼梦	9787506336239	1

CASE WHEN ... THEN ... END 它根据指定的条件返回不同的值。在这种情况下，条件是借阅.罚款分类号 IS NOT NULL。

14、 查询借阅及罚款分类信息，如果有罚款则显示借阅信息及罚款名称、罚金，如果没有罚款则罚款名称、罚金显示空（左外连接）

```
SELECT 借阅.*, 罚款分类.罚款名称, 罚款分类.罚金
FROM 借阅
LEFT JOIN 罚款分类 ON 借阅.罚款分类号 = 罚款分类.罚款分类号;
```

```
SELECT 借阅.*, 罚款分类.罚款名称, 罚款分类.罚金
FROM 借阅
LEFT JOIN 罚款分类 ON 借阅.罚款分类号 = 罚款分类.罚款分类号;
```

	借阅流水号	借书证号	图书编号	借书日期	归还日期	罚款分类号	备注	罚款名称	罚金
1	1	20081237	3007071	2010/09/19	2010/09/20		...		
2	2	20071235	1005063	2010/10/20	2011/02/20	1	...	延期	10.00
3	3	20071235	2001232	2011/09/01			...		
4	4	20061234	1005063	2011/09/20			...		
5	5	20051001	3007071	2011/09/10			...		
6	6	20071235	1005050	2011/10/20	2012/02/20	1	...	延期	10.00

LEET JOIN 连接两个表，但结果集会包括左侧表的所有记录，即使右侧表为空。

15、 查询借阅了所有“文学”类书目的读者的姓名、单位。

```
SELECT DISTINCT 读者.姓名, 读者.单位
```

```
FROM 读者
```

```
JOIN 借阅 ON 读者.借书证号 = 借阅.借书证号
```

```
JOIN 图书 ON 借阅.图书编号 = 图书.图书编号
```

```
JOIN 书目 ON 书目.ISBN = 图书.ISBN
```

```
JOIN 图书分类 ON 书目.图书分类号 = 图书分类.图书分类号
```

```
WHERE 图书分类.类名 = '文学'
```

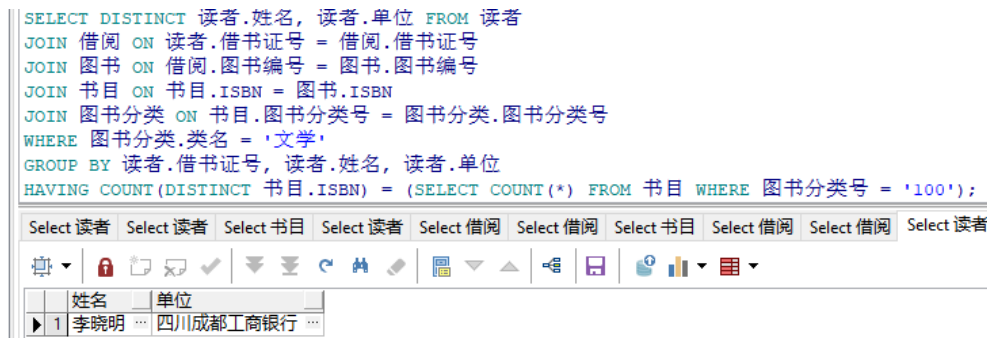
```
GROUP BY 读者.借书证号, 读者.姓名, 读者.单位
```

```
HAVING COUNT(DISTINCT 书目.ISBN) = (SELECT COUNT(*) FROM 书
```

```
目 WHERE 图书分类号 = '100');
```

--确保借阅了所有文学类书目的读者才会被选中。如果“文学”类别中有 N

本书，那么只有借阅了全部 N 本书的读者才会出现在结果集中。



为什么：

4. 实验代码

```
SELECT 图书.图书编号,书目.ISBN,书目.书名,书目.作者,书目.出版单位 FROM
图书
JOIN 书目 ON 书目.ISBN = 图书.ISBN
WHERE 图书.是否借出 = '否' AND 书目.书名 = '红楼梦';
```

```
SELECT ISBN, 书名, 单价 FROM 书目
WHERE 出版单位 = '高等教育出版社'
ORDER BY 单价 DESC;
```

```
SELECT ISBN, COUNT(*) AS 藏书数量
FROM 书目
WHERE 书名 = '红楼梦'
GROUP BY ISBN;
```

```
SELECT 图书.图书编号, 图书.ISBN, 书目.书名 FROM 借阅
JOIN 图书 ON 图书.图书编号 = 借阅.图书编号
JOIN 书目 ON 书目.ISBN = 图书.ISBN
WHERE 借阅.借书证号 = '20061234' AND 借阅.归还日期 IS NULL;
```

```
SELECT 出版单位, MAX(单价) AS 最高单价, AVG(单价) AS 平均单价
FROM 书目
GROUP BY 出版单位;
```

```
SELECT 读者.借书证号, 读者.姓名, 读者.单位, 读者.性别, 读者.地址, 读者.联
系电话, 读者.身份证编号, COUNT(借阅.借阅流水号) AS 借阅次数
FROM 读者
JOIN 借阅 ON 借阅.借书证号 = 读者.借书证号
GROUP BY 读者.借书证号, 读者.姓名, 读者.单位, 读者.性别, 读者.地址, 读者.
联系电话, 读者.身份证编号
HAVING COUNT(借阅.借阅流水号) >= 2;
```

```
SELECT 读者.单位, 书目.书名, 借阅.借书日期 FROM 读者
JOIN 借阅 ON 借阅.借书证号 = 读者.借书证号
JOIN 图书 ON 借阅.图书编号 = 图书.图书编号
JOIN 书目 ON 书目.ISBN = 图书.ISBN
WHERE 读者.姓名 = '王菲';
```

```
SELECT 图书分类号, COUNT(*) AS 册数, AVG(单价) AS 平均单价
FROM 书目
GROUP BY 图书分类号;
```

```
SELECT COUNT(*)
FROM 读者
WHERE 借书证号 NOT IN (SELECT DISTINCT 借书证号 FROM 借阅);
```

```
SELECT COUNT(DISTINCT 借书证号)
FROM 借阅;
```

```
SELECT 读者.借书证号, 读者.姓名, 读者.单位, 读者.性别, 读者.地址, 读者.联系电话, 读者.身份证编号, 图书.图书编号, 书目.书名
FROM 借阅
JOIN 读者 ON 借阅.借书证号 = 读者.借书证号
JOIN 图书 ON 借阅.图书编号 = 图书.图书编号
JOIN 书目 ON 书目.ISBN = 图书.ISBN
WHERE 借阅.归还日期 IS NULL;
```

```
SELECT 书名, 作者
FROM 书目
WHERE 书名 LIKE 'Internet%';
```

```
SELECT 书目.书名, 书目.ISBN, COUNT(CASE WHEN 借阅.罚款分类号 IS NOT
NULL THEN 1 END) AS 罚款次数
FROM 借阅
JOIN 图书 ON 借阅.图书编号 = 图书.图书编号
JOIN 书目 ON 书目.ISBN = 图书.ISBN
GROUP BY 书目.ISBN, 书目.书名;
```

```
SELECT 借阅.*, 罚款分类.罚款名称, 罚款分类.罚金
FROM 借阅
LEFT JOIN 罚款分类 ON 借阅.罚款分类号 = 罚款分类.罚款分类号;
```

```
SELECT DISTINCT 读者.姓名, 读者.单位 FROM 读者
JOIN 借阅 ON 读者.借书证号 = 借阅.借书证号
JOIN 图书 ON 借阅.图书编号 = 图书.图书编号
JOIN 书目 ON 书目.ISBN = 图书.ISBN
```

```
JOIN 图书分类 ON 书目.图书分类号 = 图书分类.图书分类号
WHERE 图书分类.类名 = '文学'
GROUP BY 读者.借书证号, 读者.姓名, 读者.单位
HAVING COUNT(DISTINCT 书目.ISBN) = (SELECT COUNT(*) FROM 书目
WHERE 图书分类号 = '100');
```

扩展:

```
--ALTER TABLE 书目 ADD (出版年份 NUMBER);
UPDATE 书目 SET 出版年份 = 2005 WHERE ISBN = '7040195836';
UPDATE 书目 SET 出版年份 = 1983 WHERE ISBN = '9787508040110';
UPDATE 书目 SET 出版年份 = 2008 WHERE ISBN = '9787506336239';
UPDATE 书目 SET 出版年份 = 2009 WHERE ISBN = '9787010073750';
SELECT * FROM 书目;
```

```
SELECT COUNT(*) AS 总藏书量, SUM(单价) AS 藏书总金额, MAX(单价) AS
最高价, MIN(单价) AS 最低价
FROM 书目;
```

```
SELECT 书目.书名, 书目.作者, 书目.出版单位, 书目.出版年份
FROM 书目
JOIN 图书 ON 书目.ISBN = 图书.ISBN
GROUP BY 书目.ISBN, 书目.书名, 书目.作者, 书目.出版单位, 书目.出版年份
HAVING COUNT(图书.ISBN) >= 5;
```

```
SELECT * FROM 书目 WHERE 出版年份 = (SELECT MIN(出版年份) FROM
书目);
```

```
SELECT COUNT(*) FROM 图书 WHERE 是否借出 = '是';
```

```
SELECT 出版年份, COUNT(*) AS 册数 FROM 书目
GROUP BY 出版年份
ORDER BY 册数 DESC FETCH FIRST 1 ROWS ONLY;
```

```
SELECT 借书证号, COUNT(*) AS 未归还册数 FROM 借阅
WHERE 归还日期 IS NULL
GROUP BY 借书证号
ORDER BY 未归还册数 DESC FETCH FIRST 1 ROWS ONLY;
```

```
SELECT AVG(借书册数)
FROM (SELECT 借书证号, COUNT(*) AS 借书册数
      FROM 借阅
      GROUP BY 借书证号);
```

```

SELECT 单位, AVG(借书册数) AS 平均借书册数
FROM (SELECT 读者.借书证号, 读者.单位, COUNT(*) AS 借书册数
      FROM 借阅
      JOIN 读者 ON 借阅.借书证号 = 读者.借书证号
      GROUP BY 读者.借书证号, 读者.单位)
GROUP BY 单位
ORDER BY 平均借书册数 DESC FETCH FIRST 1 ROWS ONLY;

```

```

SELECT 书目.ISBN, 书目.书名, 书目.作者, 书目.出版单位, 书目.出版年份, 书
目.单价, 书目.图书分类号
FROM 书目
WHERE 书目.ISBN NOT IN (
    SELECT 图书.ISBN
    FROM 图书
    JOIN 借阅 ON 图书.图书编号 = 借阅.图书编号
    WHERE 借阅.借书日期 BETWEEN TO_DATE('2022/10/26',
'YYYY/MM/DD') AND TO_DATE('2024/10/26', 'YYYY/MM/DD'));

```

```

SELECT 书目.ISBN, 书目.书名, 书目.作者, 书目.出版单位, 书目.出版年份, 书
目.单价, 书目.图书分类号
FROM 书目
WHERE 书目.ISBN NOT IN (
    SELECT 图书.ISBN
    FROM 图书
    JOIN 借阅 ON 图书.图书编号 = 借阅.图书编号
    WHERE 借阅.借书日期 BETWEEN TRUNC(sysdate) - INTERVAL '2'
YEAR AND TRUNC(sysdate)
);

```

```

SELECT 借书证号 FROM 借阅
WHERE 借书日期 NOT BETWEEN TRUNC(sysdate, 'YEAR') AND
TRUNC(sysdate, 'YEAR') + INTERVAL '1' YEAR;

```

6. 实验结果

3.2.6 实验扩展

1、在书目关系中新增“出版年份”，并在该属性下添加数据。（使用 SQL 完成）



ISBN	书名	作者	出版单位	出版年份	单价	图书分类号
7040195836	数据库系统概论	王珊	高等教育出版社	2005	39.00	200
9787508040110	红楼梦	曹雪芹	人民出版社	1983	20.00	100
9787506336239	红楼梦	曹雪芹	作家出版社	2008	34.30	100
9787010073750	心学之路	张立文	人民出版社	2009	33.80	300

ALTER TABLE 书目 ADD (出版年份 NUMBER);

在书目表中添加一个新列，列名为出版年份，数据类型为 NUMBER（存储整数）

UPDATE 书目 SET 出版年份 = 2005 WHERE ISBN = '7040195836';

UPDATE 书目 SET 出版年份 = 1983 WHERE ISBN = '9787508040110';

UPDATE 书目 SET 出版年份 = 2008 WHERE ISBN = '9787506336239';

UPDATE 书目 SET 出版年份 = 2009 WHERE ISBN = '9787010073750';

```
--ALTER TABLE 书目 ADD (出版年份 NUMBER);
UPDATE 书目 SET 出版年份 = 2005 WHERE ISBN = '7040195836';
UPDATE 书目 SET 出版年份 = 1983 WHERE ISBN = '9787508040110';
UPDATE 书目 SET 出版年份 = 2008 WHERE ISBN = '9787506336239';
UPDATE 书目 SET 出版年份 = 2009 WHERE ISBN = '9787010073750';
SELECT * FROM 书目;
```

ISBN	书名	作者	出版单位	单价	图书分类号	出版年份
7040195836	数据库系统概论	王珊	高等教育出版社	39.00	200	2005
9787508040110	红楼梦	曹雪芹	人民出版社	20.00	100	1983
9787506336239	红楼梦	曹雪芹	作家出版社	34.30	100	2008
9787010073750	心学之路	张立文	人民出版社	33.80	300	2009

2、求总藏书量、藏书总金额、最高价、最低价。

SELECT COUNT(*) AS 总藏书量, SUM(单价) AS 藏书总金额, MAX(单价)

AS 最高价, MIN(单价) AS 最低价

FROM 书目;

<pre>SELECT COUNT(*) AS 总藏书量, SUM(单价) AS 藏书总金额, MAX(单价) AS 最高价, MIN(单价) AS 最低价 FROM 书目;</pre>				
Update 书目 Update 书目 Update 书目 Update 书目 Select 书目 Select 书目				
	总藏书量	藏书总金额	最高价	最低价
▶ 1	4	127.1	39	20

3、列出藏书在 5 本以上的书目（书名、作者、出版社、出版年份）。

```
SELECT 书目.书名, 书目.作者, 书目.出版单位, 书目.出版年份
```

```
FROM 书目
```

```
JOIN 图书 ON 书目.ISBN = 图书.ISBN
```

```
GROUP BY 书目.ISBN, 书目.书名, 书目.作者, 书目.出版单位, 书目.出版年
```

```
份
```

```
HAVING COUNT(图书.ISBN) >= 5;
```

<pre>SELECT 书目.书名, 书目.作者, 书目.出版单位, 书目.出版年份 FROM 书目 JOIN 图书 ON 书目.ISBN = 图书.ISBN GROUP BY 书目.ISBN, 书目.书名, 书目.作者, 书目.出版单位, 书目.出版年份 HAVING COUNT(图书.ISBN) >= 5;</pre>				
	书名	作者	出版单位	出版年份

HAVING 子句用于对分组后的结果进行条件筛选，常和聚合函数一起使用。

4、列出年份最久远的书？

```
SELECT * FROM 书目 WHERE 出版年份 = (SELECT MIN(出版年份)
```

```
FROM 书目);
```


SELECT * FROM 书目 WHERE 出版年份 = (SELECT MIN(出版年份) FROM 书目);							
Update 书目 Update 书目 Update 书目 Update 书目 Select 书目 Select 书目 Select 书目 Select 书目							
	ISBN	书名	作者	出版单位	单价	图书分类号	出版年份
1	9787508040110	红楼梦	曹雪芹	人民出版社	20.00	100	1983

5、 目前实际已借出多少册书?

SELECT COUNT(*) FROM 图书 WHERE 是否借出 = '是';

SELECT COUNT(*) FROM 图书 WHERE 是否借出 = '是';							
Update 书目 Update 书目 Update 书目 Update 书目 Select 书目 S							
	COUNT(*)						
1	4						

6、 哪一年的图书最多?

SELECT 出版年份, COUNT(*) AS 册数 FROM 书目

GROUP BY 出版年份

ORDER BY 册数 DESC

FETCH FIRST 1 ROWS ONLY; --用于限制查询结果只返回排序后的第一条

记录

SELECT 出版年份, COUNT(*) AS 册数 FROM 书目 GROUP BY 出版年份 ORDER BY 册数 DESC FETCH FIRST 1 ROWS ONLY;							
Update 书目 Update 书目 Update 书目 Update 书目 Select =							
	出版年份	册数					
1	2009	1					

7、 哪本借书证未归还的图书最多?

```
SELECT 借书证号, COUNT(*) AS 未归还册数 FROM 借阅
```

WHERE 归还日期 IS NULL

GROUP BY 借书证号

ORDER BY 未归还册数 DESC FETCH FIRST 1 ROWS ONLY;

```
SELECT 借书证号, COUNT(*) AS 未归还册数 FROM 借阅
WHERE 归还日期 IS NULL
GROUP BY 借书证号
ORDER BY 未归还册数 DESC FETCH FIRST 1 ROWS ONLY;
```

Update 书目 Update 书目 Update 书目 Update 书目 Select 书目 Se

	借书证号	未归还册数
▶ 1	20061234	1

8、平均每本借书证的借书册数。

SELECT AVG(借书册数)

FROM (SELECT 借书证号, COUNT(*) AS 借书册数

FROM 借阅

GROUP BY 借书证号);

```
SELECT AVG(借书册数)
FROM (SELECT 借书证号, COUNT(*) AS 借书册数
      FROM 借阅
      GROUP BY 借书证号);
```

Update 书目 Update 书目 Update 书目 Select 书目 Select 书目

AVG(借书册数)

▶ 1	1.5
-----	-----

9、哪个单位的读者平均借书册数最多？

```
SELECT 单位, AVG(借书册数) AS 平均借书册数

FROM (SELECT 读者.借书证号, 读者.单位, COUNT(*) AS 借书册数

      FROM 借阅

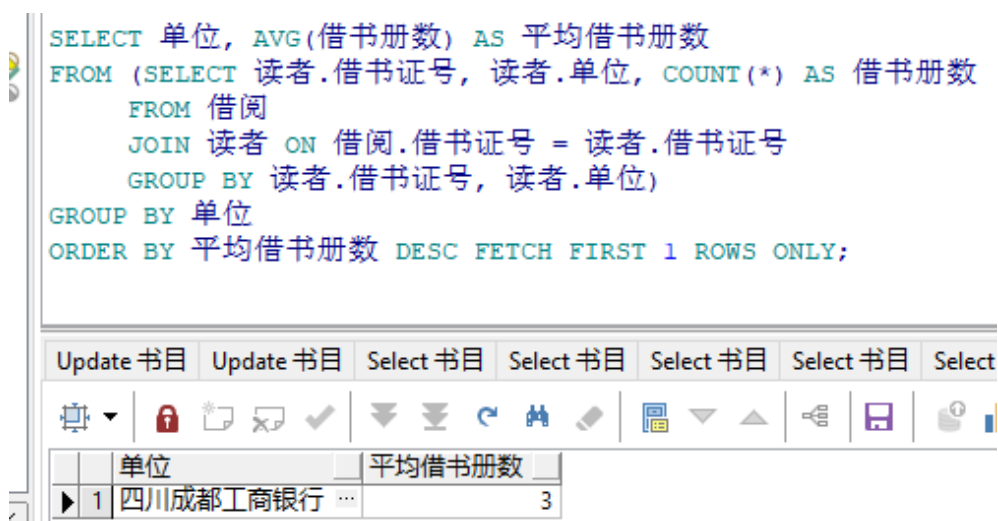
      JOIN 读者 ON 借阅.借书证号 = 读者.借书证号

      GROUP BY 读者.借书证号, 读者.单位)

GROUP BY 单位

ORDER BY 平均借书册数 DESC

FETCH FIRST 1 ROWS ONLY;
```



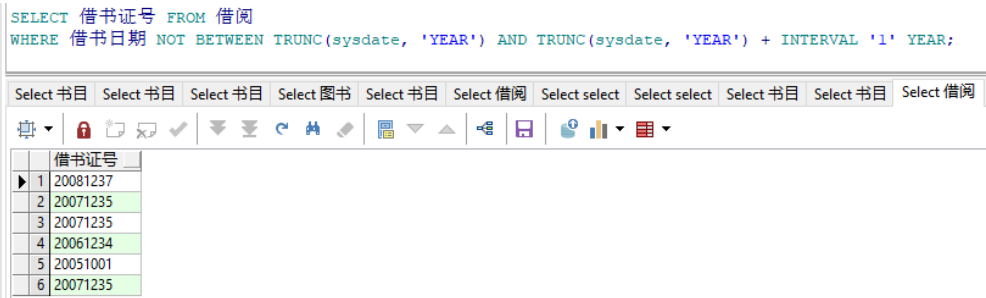
10、最近两年都未被借过的书。

一开始使用的是 to_date, 以现在的时间为基准。

11、今年未借过书的借书证。

```
SELECT 借书证号 FROM 借阅

WHERE 借书日期 NOT BETWEEN TRUNC(sysdate, 'YEAR') AND
TRUNC(sysdate, 'YEAR') + INTERVAL '1' YEAR;
```



7. 实验心得

在数据库管理与操作的学习过程中，查询操作是最核心的部分之一。通过使用 SQL 语言中的 SELECT 语句，我们能够从庞大的数据集中检索出所需的信息，包括对数据的基本检索、对数据的筛选、分组、聚合和排序等高级操作。

在单表查询中，我们通常关注于单一数据表中的信息提取。通过精确的 **WHERE 子句** 条件，我们可以筛选出符合特定标准的记录。

当我们的数据分布在多个表中时，可以通过 **JOIN 操作** 能够将不同表中的数据根据一定的关联条件合并在一起，从而得到更加完整和丰富的信息。

嵌套查询，又称为子查询，允许我们将一个 SELECT 语句嵌入到另一个 SELECT 语句中。这种方式适用于那些需要根据另一个查询结果来决定当前查询条件的场景。

集合查询则涉及到多个 SELECT 语句的组合使用,如 **UNION** 和 **INTERSECT** 等操作。这些操作允许我们将多个查询的结果集合在一起,或者找出它们之间的共同点。这种查询方式在需要比较不同数据集时非常有用。

通过这些查询操作的学习,我深刻体会到了数据库查询的强大能力和灵活性。每一次成功的查询都需要仔细分析数据结构,合理设计查询条件,从复杂的数据中提取出有价值的信息。